Templates

A MINIMAL APPLICATION WITH THE COMPLETE FRAMEWORK STRUCTURE

START, EXTEND, TEST AND OPERATE

Developer and Operator Guide

Identity, a visual SmartList and ordinary chat, using the same development and deployment foundations as MyScoutee.

DOCUMENT ID	DOCUMENT VERSION
T-START-001	1.4.0
SOFTWARE VERSION	**RELEASED**
1.4.0	2026-10-12
STATUS	
Implementation qualification in progress	

Document control

This HTML file is the canonical manual. Generate the single stable PDF path with `node guides/manuals/build-manuals.mjs T-START-001`. Git records revisions; PDF filenames do not contain a changing version. The document is not evidence of an installed or published release.

The numbered plan in `guides/plans/MSC-122_TEMPLATE_IMPLEMENTATION_PLAN.md` and the QA evidence identify passed, failed and pending gates. Local VMware installation qualification is in progress. Netcup is outside the authorized work: do not connect to or change it.

Contents

Framework and domain selection

Checkout and frontend ownership

Local and HTTP development

Automated qualification

KVM and monitoring

Data and account lifecycle

Build another application

Installed operation and releases

1. Framework and domain selection

The landing page shows Templates, login and the version badge. Members have a visual sample-card SmartList and ordinary chat. The avatar menu retains the small profile and image editor, API access and minimal settings. Notifications, guides, help documents and consent remain available. The administrator shell has no product navigation; administrator API access remains. Operator configuration, email, TLS and update surfaces are retained.

Chat includes message editing, deletion, replies, reactions, pins, read state, reconnect and two-party audio/video calls. It excludes MyScoutee invitations, reports, feedback and support-case management. Calls use encrypted WebRTC media and STUN discovery. A local fake-camera test does not prove physical devices, public-network traversal or a TURN relay.

There is no event, community/group, matchmaking, payment or virtual-company Registry implementation. Necessary account and chat model fields retain their original meaning. Optional product fields are omitted from JSON rather than recreated as empty product domains. Localization and geographic location are both retained.

2. Checkout and frontend ownership

The backend is the parent repository. Its `frontend` submodule is pinned to myscoutee-template-frontend. Run `tools/bootstrap-submodules.sh` after cloning the parent to initialize the recorded frontend and shared `components` commits.

Commit and push frontend changes to its own repository first, then commit the updated gitlink in the parent. The bootstrap refuses a dirty frontend. Both submodules use ordinary GitHub repository URLs.

```
git clone https://github.com/fssrepository/myscoutee-template.git
cd myscoutee-template
bash tools/bootstrap-submodules.sh
cp server/.env.example server/.env
npm --prefix frontend ci
```

Use the checked-in Gradle wrapper and frontend lockfile. The development environment uses Java 25, Node 24, Docker Compose and a supported Chrome/Chromium. Install frontend dependencies with `npm ci` in `frontend` after initializing components. Never copy local credentials, generated databases or build caches into Git.

3. Local and HTTP development

Browser-local mode uses IndexedDB and the selected four-person seed. In `frontend`, run `npm run start:local` and open `http://127.0.0.1:4200`. It is a UI/data-adapter environment, not a backend or live WebRTC deployment.

HTTP development uses the versioned `server/docker-compose-dev.yml` and `server/docker-dev.env`. Its existing backend hot-deploy and frontend synchronization process own compilation and asset delivery. Inspect that process and its logs before starting a build. Do not start another watcher or compile the same target alongside it.

The frontend keeps distinct local, dev, E2E, production and GitHub Pages configurations. Production output belongs to `frontend/prod`; Pages output belongs to `frontend/docs`. Frontend synchronization, runtime availability and build stamping remain part of the workflow. Preserve relative assets and the configured base path when changing the application name.

Stop the active dev stack before starting KVM, E2E or another application stack. A monolith and the independent-service lab are alternate deployments, not parallel development servers.

4. Automated qualification

`tools/run-backend-junit.sh standard` runs the selected Java unit tests with isolated outputs. Use its `--tests` filter for a focused change; embedded Mongo checks use the separate `embedded` group. Frontend tests use `npm test -- --watch=false`. Do not rerun every suite after each small edit.

`e2e/README.md` defines the disposable E2E composition, certificate trust, fixture variables and serial runners. Run `node e2e/functional/backend/run.mjs` for the retained API contracts and `node e2e/functional/ui/run.mjs --fail-fast` for browser flows. `--case` selects one matching flow. Each fixture owns its actors and removes them afterward; it must not reset production data or another user's sessions.

Record actual build revisions, image IDs, environment and fixture scope. Browser-local success does not qualify HTTP, and a healthy container does not prove a working user flow. Compare CPU and latency using identical workloads and limits after a coherent change batch. The seven-case read workload has a six-user, 60-second baseline and repeat with zero errors. Throughput was 112.84 and 114.92 completed collection cases/second; p95 was 62.69 and 58.34 ms. These are repeatability results, not an improvement claim against MyScoutee. Browser/WebSocket load measurements remain pending.

```
export E2E_NATIVE_DEMO_SESSIONS=true E2E_SKIP_E2E_RESET=true
export E2E_STRESS_BASE_URL=https://localhost
export NODE_EXTRA_CA_CERTS=/path/to/local-ca.pem
node e2e/stress/backend/run.mjs --scenario template-read --duration-ms 15000
node e2e/stress/backend/run.mjs --scenario template-read --record
```

See `e2e/stress/README.md` for matched comparisons and the separate KVM CPU measurements.

5. KVM and monitoring

The template owns a separate VM named `myscoutee-template`, with four vCPUs and 6 GiB RAM. Its private keys, disks and Terraform state live outside Git under `~/.local/state/myscoutee-template/microservices`. Host-boot autostart is disabled. The namespace contains four JVM owners:

Owner	Selected domains
identity	Accounts, access, minimal administration, platform, integrations and showcase
communication	Chat, notifications, email and localization

Owner	Selected domains
media	Media storage and processing
help	Help, guide and consent documents

The lab includes a two-member Mongo replica set with distinct REAL and DEMO databases, Redis, MinIO, MCP, STUN, ingress and a local release simulator. Two voting Mongo members require both for majority writes; this is not one-node-loss availability.

```
microservices/tools/lab.sh preflight
microservices/tools/lab.sh build
microservices/tools/lab.sh frontend-image
python3 microservices/tools/workloads.py lock-images
python3 microservices/tools/vm.py start
python3 microservices/tools/workloads.py backends-stop
python3 microservices/tools/workloads.py up
python3 microservices/tools/workloads.py forward
microservices/tools/lab.sh monitoring-up
```

Build support images when their canonical sources change using `lab.sh support-images`, `minio-image` and `release-simulator-image`. Startup imports locked images; it does not silently rebuild. Stop backends before deploying changed writers. `seed-mongo.py` owns Mongo initialization; do not replace it with hand-written import commands.

The application is at `http://localhost:18080`, Grafana at `http://localhost:13000`, and Prometheus at `http://localhost:19090`. The loopback-only Grafana login is `lab / template-local-only`. Use `microservices/tools/open-terminals.sh` for the live status view or `python3 microservices/tools/vm.py observe --once` for one snapshot. The terminal shows guest/container CPU, RAM, readiness and host monitoring. Closing it does not stop the VM.

```
microservices/tools/lab.sh monitoring-stop
python3 microservices/tools/vm.py stop
```

These commands retain owned data and do not enable autostart. Use the versioned ownership-checked destroy tools only when intentionally discarding a lab; never use global Docker prune as a cleanup substitute.

6. Data and account lifecycle

Canonical Mongo collection/index definitions, selected system catalogs and small demo fixtures are under `server/docker/conf/mongodb`. Production identity records are empty. The operator provisions real credentials and branding after installation. Browser-local seeds use the same selected model meanings.

Account deletion keeps the original grace period, normally 30 days, followed by durable purge retries across the retained owners. Notification outbox delivery, scheduled messages, warning email and integration callback work retain their own state and retry behavior. A removed administrator screen is not a reason to remove necessary lifecycle jobs.

Keep REAL and DEMO boundaries intact in HTTP, internal transport, workers and sockets. Internal actor grants are short-lived and validated per operation. A persistent split-service socket renews through its original account session; message bursts reuse its current grant. Never replace session checks with a caller-supplied user ID.

7. Build another application

Start with the showcase owner and its visual cards. Replace those sample records and presentations with your domain while keeping identity, navigation, popup behavior, SmartList views and chat infrastructure. Add optional JSON fields only when the new application needs them.

Canonical Java implementations remain in `server/projects/profile`. Add a domain's contracts and implementation there, then select its owner in `microservices/assemblies.json`. Cross-owner calls use the existing typed transport and authority protocol. Do not copy the monolith into each service.

Frontend contracts, services and repositories follow domain folders across base, HTTP and local adapters. Extend the existing component/store/repository owner, preserving observable local/HTTP behavior. Shared visual controls belong to the existing components submodule. `tools/upstream-sources.json` records the source revisions and deliberate adaptations needed for future upgrades.

8. Installed operation and releases

The Debian package is `myscoutee-template`, version 1.4.0. It contains the five managed images, selected system/demo seeds, systemd units and common host helpers. MongoDB, Redis and STUN use the pinned upstream images. Package files live under `/opt/myscoutee-template`, deployment configuration under `/etc/myscoutee-template`, state under `/var/lib/myscoutee-template`, and backups under `/var/backups/myscoutee-template`.

The shared installer generates fresh MongoDB/MinIO credentials, an encryption master key and a deployment-local Operator login. Existing credentials, keys and manual TLS files are preserved. Firebase starts disabled; enabling it requires the deployment service-account file selected by Compose. The installer never creates a Registry identity or imports payment configuration.

Host-script tests cover package identity, image selection, generated secrets, TLS preservation, update ingress, purge isolation and failed/successful restore orchestration using disposable files and command doubles. The production images passed real HTTP and browser checks. These are separate from actual DEB installation: **local VMware installation, installed update/rollback and recovery remain untested until the attended session.** Netcup must not be used.

Verify and install

Download the DEB, client-tools archive, signed release manifest, `myscoutee-update.json`, public key and checksums from the versioned GitHub release. Do not mix assets from different builds. Unpack the client tools, copy `install.env.example` to a private `install.env`, and set the local VMware address plus release-verification paths. Keep `ALLOW_UNSIGNED_LOCAL_PACKAGE=false`.

```
chmod 0600 install.env
./verify-release.sh ./myscoutee-template_1.4.0_amd64.deb \
```

```
  ./myscoutee-template-release_1.4.0.json ./release-public.pem \
  sha256:74255e852bf3720b81b990d0568f5b567124d6fb12335063f0309ccb522c5c03
./install.sh ./myscoutee-template_1.4.0_amd64.deb
```

Confirm the complete fingerprint through an independent trusted channel; a public key downloaded beside an artifact does not establish its authenticity. The current release authority is for this template distribution. A derived project must establish its own release signing key.

The client invokes package-owned host qualification. Check public transport separately from the client machine:

```
NODE_EXTRA_CA_CERTS=/path/to/local-ca.pem node verify-deployment/run.mjs \
  --url https://your-local-vm --expected-version 1.4.0 --demo-session
```

Operate and recover

```
sudo systemctl start myscoutee-template.service
sudo /opt/myscoutee-template/packaging/scripts/status-production.sh
sudo /opt/myscoutee-template/packaging/scripts/qualify-deployment.sh check \
  --expected-version 1.4.0
sudo systemctl stop myscoutee-template.service
```

TLS requests and renewal use the retained privileged worker and systemd units. Configure the public URL, certificate and Firebase provider through the existing Operator/deployment workflow. The initial localhost certificate is for bootstrap, not public certificate-chain qualification.

```
sudo env MONGO_DATABASE=production_db MYSCOUTEE_BACKUP_REQUIRE_COMPLETE=true \
  /opt/myscoutee-template/packaging/scripts/backup-production.sh
```

A complete backup contains MongoDB, configuration, the matching configuration encryption key and MinIO media. It contains secrets and remains mode 0600. Test restoration on the disposable VMware deployment before relying on it. Restore is explicitly destructive and requires the matching database/key pair:

```
sudo env CONFIRM_RESTORE=true MONGO_DATABASE=production_db \
  RESTORE_CONFIG=true RESTORE_MINIO=true \
  /opt/myscoutee-template/packaging/scripts/restore-production.sh \
  /var/backups/myscoutee-template/<backup-name>.tar.gz
```

A failed MongoDB replacement leaves the backend stopped and does not activate the backup encryption key. Ordinary package removal retains deployment data and TLS material. Explicit purge removes this package's state and managed images; external backup archives remain.

Build and sign another release

Use `packaging/release/build-production.sh` followed by `BUILD_ARTIFACTS=false bash packaging/release/package-production.sh`. The existing managed-image policy, install defaults, recovery extension, host checks and release-seed selection own project-specific choices. The template does not install development/test scripts. Do not copy an entire product domain to satisfy a packaging input.

The original pinned MinIO upstream tag became unavailable. An identical amd64 base is preserved at `ghcr.io/fssrepository/myscoutee-minio-base`, digest `sha256:52dfd5c0bbd38d3219f2058c7af216d9f9a27a994b7b5baad09bbd38866015ff`. Building from it requires registry access. The five managed images have been published privately and pulled back with matching image IDs. Builds, tests, signing and publication run locally; GitHub Actions is not required. Registry authentication is required for these private images.

`prepare-release.py` takes an existing private key, the actual DEB and client archive. It signs both, creates matching signed update metadata, verifies signatures and hashes, and records the exact upload set in `RELEASE.json`. Publish the complete set together; never publish a lone DEB. Private keys belong outside the repository and deployment host. The updater retains strict package/image/version/key checks; installed update and rollback qualification is still pending.

Build Pages locally with `MYSCOUTEE_VERSION=1.4.0 npm --prefix frontend run build:github`. Commit the generated `docs/` directory in the frontend repository. The frontend Pages workflow publishes `docs/` after every push to `master`, and can also be started manually from Actions. In its Pages settings, select `GitHub Actions`. The build includes `.nojekyll`; no separate Pages branch is needed. Build, test and signing credentials stay on the development machine.

The GitHub Pages demo uses browser-local data and shows its application version. Live messaging and calls require a backend deployment.